

Reporte de la Actividad 5: Selección de la mediana

Equipo 9 — TC2038.602

Implementación y análisis de tres algoritmos para obtener la mediana de un arreglo desordenado sin usar bibliotecas estándar de ordenamiento. Código completo en: <https://github.com/ppgrio/TC2038.602-Equipo-9>

1. Problema y algoritmos

Dado un arreglo de n enteros, encontrar el elemento de rango central. Para n impar, la mediana es el elemento en la posición $(n+1)/2$; para n par, el promedio de los dos centrales. La actividad usa tamaños impares entre 100,001 y 1,000,001, con cinco repeticiones y reordenamiento aleatorio.

Ordena + selecciona. Ordena todo el arreglo y toma el elemento central. Simple, pero realiza trabajo extra al producir un orden completo.

Quickselect con Lomuto. Elige un pivote, particiona el arreglo y continúa solo en la parte que contiene el rango buscado. Evita ordenar la otra mitad, pero su rendimiento depende de la calidad del pivote.

Mediana de medianas (BFPRT). Divide en grupos de cinco, calcula la mediana de cada grupo, elige como pivote la mediana de esas medianas, particiona y continúa en una sola parte. Está diseñado para garantizar progreso suficiente incluso en el peor caso.

2. Complejidad del código y mantenimiento

Algoritmo	Tamaño relativo	Mantenimiento
Ordena + selecciona	Bajo	Muy sencillo; delega el ordenamiento y solo accede al centro.
Lomuto	Medio	Requiere cuidar índices, límites y pivote; errores sutiles pueden degradar rendimiento
BFPRT	Alto	Incluye agrupación, medianas auxiliares, recursión y partición; más casos frontera

La solución incluye una aplicación Flask/HTML que separa la lógica algorítmica de la interfaz, pero añade rutas y plantillas. Conviene mantener pruebas unitarias para validar los tres resultados sobre los mismos datos.

3. Análisis asintótico

Ordena + selecciona. El costo dominante es el ordenamiento, $\Theta(n \log n)$; el acceso posterior es $\Theta(1)$. Total: $\Theta(n \log n)$.

Lomuto. Cada partición examina $\Theta(n)$ elementos del segmento actual. Con pivotes razonables, la suma de tamaños es $\Theta(n)$, por lo que el caso promedio es $\Theta(n)$. Si el pivote queda repetidamente en un extremo, la recurrencia $T(n)=T(n-1)+\Theta(n)$ produce $\Theta(n^2)$. Espacio: $\Theta(1)$ auxiliar más la pila recursiva.

BFPRT. Formar grupos y particionar cuesta $\Theta(n)$; la selección recursiva de medianas cuesta $T(n/5)$; el subproblema restante queda acotado por $\approx 7n/10$. La recurrencia $T(n) \leq T(n/5)+T(7n/10)+\Theta(n)$ se resuelve como $\Theta(n)$, tanto en promedio como en el peor caso.

Algoritmo	Promedio	Peor caso	Espacio auxiliar
Ordena + selecciona	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$ si copia
Lomuto	$\Theta(n)$	$\Theta(n^2)$	$\Theta(1)$ +pila
BFPRT	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$ o menor

4. Benchmark y discusión

El protocolo mide solo el cálculo: arreglos impares entre 100,001 y 1,000,001 elementos, cinco repeticiones por tamaño y shuffle antes de cada prueba. El gráfico debe incluir una serie por algoritmo, con n en el eje horizontal y

el tiempo promedio en el vertical. Debe insertarse aquí el gráfico generado por el programa de pruebas.

Se espera que ordenar y seleccionar crezca más rápido que los dos métodos lineales. Quickselect suele ser competitivo y, con datos aleatorios, puede superar a BFPRT por tener menos trabajo constante. BFPRT puede mostrar tiempos mayores para tamaños moderados debido a calcular medianas de grupos y realizar más pasos, aunque su crecimiento queda protegido frente a pivotes adversos.

5. Conclusiones

Ordenar es la alternativa más simple de mantener, pero no la más eficiente para una sola mediana. Lomuto ofrece un equilibrio práctico entre código corto y costo lineal promedio, a cambio de un peor caso cuadrático. BFPRT proporciona la garantía teórica más sólida, $\Theta(n)$ en el peor caso, pero exige mayor cuidado de implementación y documentación.

Repositorio

<https://github.com/ppgrio/TC2038.602-Equipo-9>